

Conducción Autónoma en Duckietown mediante Aprendizaje por Refuerzo profundo

Memoria técnica — Proyecto Final MSc IA

Adolfo (AdolfoPM02)

Junio 2026

Abstract

Se entrena un agente de *Deep Reinforcement Learning* para conducir en el simulador 3D **Duckietown** usando únicamente píxeles de la cámara frontal. El trabajo cubre tres fases: (1) Q-Learning tabular *desde cero* en **FrozenLake-v1**; (2) dos baselines con Stable-Baselines3 —**DQN** (acción discreta) y **PPO** (acción continua)—; y (3) un algoritmo avanzado, **SAC**. El modelo final seleccionado es un **PPO entrenado 20 000 pasos**, que obtiene la mejor recompensa en el mapa oculto de evaluación (`loop_obstacles`) y supera ampliamente a una variante PPO de 50 000 pasos que degeneró. Se documenta además la resolución de un *segmentation fault* nativo (Duckietown + OpenGL + SB3) mediante un orden de inicialización *model-first*, clave para que el pipeline funcione en Google Colab.

1 Introducción y objetivo

El objetivo es entrenar un agente capaz de mantenerse en el carril en Duckietown a partir de observaciones de imagen, y que **generalice** a un mapa no visto durante el entrenamiento. La nota depende del desempeño en el mapa oculto `Duckietown-loop_obstacles-v0`, con obstáculos estáticos.

Contrato de evaluación. El modelo debe (i) esperar observaciones de forma $(1, 64, 64)$ apiladas en 4 frames $\rightarrow (4, 64, 64)$; (ii) llevar definidas e importables las clases personalizadas (`CustomCNN`, `wrappers`); y (iii) cargar “a la primera” en un entorno limpio reconstruido desde `requirements.txt`. Entrenar en el mapa oculto supone descalificación.

2 Fase 1: Q-Learning tabular (FrozenLake-v1)

Como calentamiento se implementa Q-Learning tabular sin librerías de Deep RL sobre FrozenLake-v1 8×8 *resbaladiza*. La tabla $Q \in \mathbb{R}^{64 \times 4}$ se actualiza con la ecuación de Bellman (TD(0)):

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)], \quad (1)$$

con política ε -greedy y decaimiento lineal de ε ($1.0 \rightarrow 0.05$), $\alpha = 0.1$, $\gamma = 0.99$ y 50 000 episodios. La política greedy aprendida alcanza una **tasa de éxito** ≈ 0.52 (frente a ≈ 0 del azar en este entorno resbaladizo). La Figura 1 muestra la curva de media móvil de recompensa. Implementación en `q_learning_sandbox.py`.

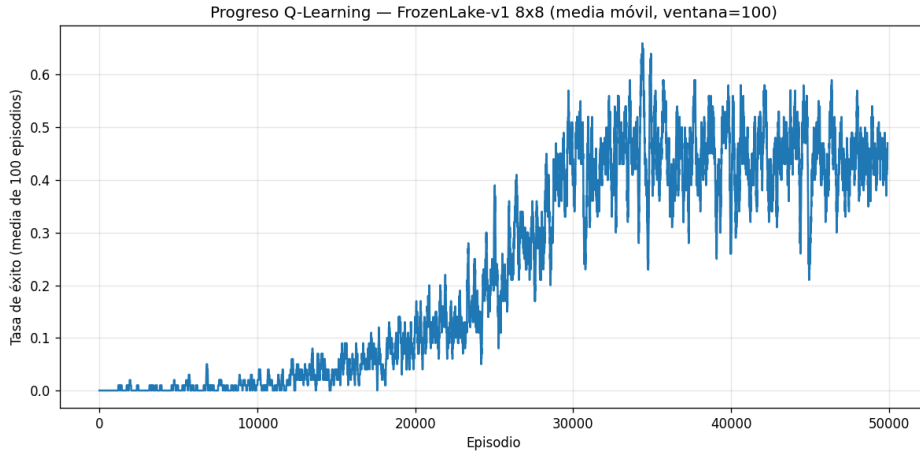


Figure 1: Fase 1: media móvil (ventana 100) de la recompensa por episodio en FrozenLake-v1 8×8 resbaladiza. El aprendizaje despegue hacia el episodio $\sim 20k$ y se estabiliza en torno a 0.45–0.50.

3 Pipeline de visión y arquitectura

El entorno real de Duckietown (API de `gym` antiguo) se adapta a Gymnasium mediante `DuckieWrapper` (`src/wrappers.py`). El procesamiento de cada observación es:

RGB ($480 \times 640 \times 3$) $\rightarrow$ recorte 50% superior (cielo) $\rightarrow$ grises $\rightarrow$ resize $64 \times 64 \rightarrow (1, 64, 64)$

y, tras `VecFrameStack(n_stack=4)` (`src/envs.py`), $(4, 64, 64)$. Las acciones continuas se normalizan a un vector plano $[v, \delta]$ con $v \in [0, 1]$, $\delta \in [-1, 1]$ (necesario: SB3 puede entregar la acción con dimensión extra, lo que rompía la dinámica de Duckietown).

CustomCNN. Extractor de características estilo *NatureCNN* (`src/cnn.py`): tres capas convolucionales $\text{Conv}(32, 8, 4) \rightarrow \text{Conv}(64, 4, 2) \rightarrow \text{Conv}(64, 3, 1)$ con ReLU, *flatten* y capa lineal a 256 características; normaliza la entrada `uint8` dividiendo por 255. Se conecta a SB3 vía `policy_kwargs` y es común a los tres algoritmos.

4 Algoritmos

DQN (baseline discreto). Q-Learning profundo *off-policy*. Como Duckietown es continuo, `DiscreteActionWrapper` expone 5 acciones discretas (recto, giros suaves y fuertes) mapeadas a comandos $[v, \delta]$. Config.: `lr`= 10^{-4} , `buffer`= 50k, `learning_starts`= 10k, `batch`= 32, `train_freq`= 4, `target_update`= 1000, exploración ε $1.0 \rightarrow 0.05$.

PPO (baseline continuo). Gradiente de política *on-policy* con *clipping*, estable y eficiente en muestras. Config.: `lr`= $3 \cdot 10^{-4}$, `n_steps`= 2048, `batch`= 64, `n_epochs`= 10, $\gamma = 0.99$, $\lambda_{\text{GAE}} = 0.95$, `clip`= 0.2.

SAC (algoritmo avanzado, Fase 3). Actor-Crítico *off-policy* de máxima entropía, con exploración regulada automáticamente. Config.: `lr`= $3 \cdot 10^{-4}$, `buffer`= 50k, `batch`= 256, $\tau = 0.005$, `ent_coef`=auto. Comparte `CustomCNN` y wrappers con PPO.

5 Metodología de entrenamiento

El entrenamiento real se realizó en **Google Colab**. El stack de Duckietown exige **Python 3.11** (incompatible con el kernel 3.12 de Colab), por lo que se monta un **venv 3.11** y se ejecuta todo como subproceso. Se resolvieron, de forma iterativa, varios fallos nativos:

- **Dependencias:** `gym-duckietown` se instala con `--no-deps` y se re-fija `numpy==1.23.5` al final, para no romper el resto del stack.
- **Render headless:** `xvfb-run + MPLBACKEND=Agg`.
- ***Segmentation fault* (Duckietown+OpenGL+SB3):** ocurría al *inicializar* el modelo con el entorno real. Un diagnóstico por aislamiento (entorno sintético con espacios idénticos) demostró que SB3/torch/CNN eran correctos y el crash dependía del entorno real. La solución es el orden ***model-first***: construir el modelo sobre un entorno sintético y luego `model.set_env(real)`. `train.py` y `eval.py` lo soportan con `--init-order model-first`.

Regla de seguridad: el mapa oculto `loop_obstacles` está bloqueado para entrenamiento (`ValueError`) y solo se usa para evaluación.

6 Resultados

Evaluación con `eval.py` (3 episodios por mapa, política determinista). La Tabla 1 compara los tres algoritmos (DQN, PPO, SAC) en sus variantes de 20k y 50k. Cada celda es *recompensa media $\pm$ std / longitud media*.

Modelo	ts	loop_empty	small_loop	loop_obstacles
DQN 20k	20k	$-1013.4 \pm 27.0 / 112$	$-1001.9 \pm 46.8 / 86$	$-1065.8 \pm 20.1 / 61$
DQN 50k	50k	$-1047.6 \pm 76.9 / 82$	$-1016.6 \pm 65.7 / 74$	$-1032.0 \pm 52.4 / 37$
PPO 20k	20k	$961.0 \pm 605.4 / 1500$	$317.5 \pm 703.3 / 1500$	$1118.2 \pm 488.2 / 1500$
PPO 50k	50k	$-813.7 \pm 210.5 / 1500$	$-1238.9 \pm 393.6 / 1500$	$-1105.4 \pm 760.6 / 1500$
SAC 20k	20k	$990.6 \pm 139.3 / 1500$	$-1253.6 \pm 120.3 / 296$	$915.2 \pm 571.9 / 1500$
SAC 50k	50k	<i>no completado (ver nota)</i>		

Table 1: Evaluación de los tres algoritmos. `loop_obstacles` es el mapa oculto, usado *solo* para evaluación (`--allow-eval-hidden`), nunca para entrenar. Longitud 1500 = el episodio alcanza el límite de pasos sin terminar antes. **Ganador: PPO 20k** (`best_agent.zip`). *ts* = timesteps.

Nota (SAC 50k). El intento de SAC a 50k se **interrumpió en Colab en torno a 30k timesteps tras más de 6h** y **no generó el artefacto `sac_advanced_50k.zip`**; por reproducibilidad y trazabilidad *no* se reporta como resultado cuantitativo, solo como intento no completado.

Dry-run del contrato. En un Colab limpio se reconstruyó el entorno desde `requirements.txt`, se instaló `gym-duckietown` aparte, se re-fijó `numpy==1.23.5` y se cargó `best_agent.zip`. Versiones verificadas:

```
numpy 1.23.5 | torch 2.12.0+cu130 | gym 0.25.2
gymnasium 0.29.1 | stable_baselines3 2.2.1 | gym_duckietown OK
```

La carga y evaluación en `loop_empty` terminó sin error (**reward 535.150, longitud 1500.0**), confirmando el “carga a la primera”.

7 Discusión

Por qué gana PPO 20k. Es el único modelo con recompensa *positiva* en los tres mapas y, sobre todo, el mejor en el mapa oculto (1118.216), lo que indica buena **generalización** y no mero sobreajuste a `loop_empty`. Su robustez relativa encaja con la naturaleza *on-policy* y estable de PPO sobre observaciones de imagen.

Por qué se descarta PPO 50k. Más pasos *no* mejoraron: las recompensas son negativas en los tres mapas pese a mantener longitud 1500. Es un caso de degradación de la política (probable colapso/entropía baja o sobre-optimización del *proxy* de recompensa), un recordatorio de que en RL más entrenamiento no implica mejor política.

DQN (baseline discreto). Tanto 20k como 50k se entrenaron y evaluaron, y se **descartan**: recompensas negativas y, sobre todo, **episodios muy cortos** (longitudes 36–112 frente a 1500), es decir, el agente sale del carril enseguida. Más entrenamiento (50k) no lo corrige.

SAC (Fase 3, algoritmo avanzado). SAC se **implementó, entrenó y evaluó** mediante **SAC 20k**: logra buen desempeño en `loop_empty` (990.6) y en el mapa oculto `loop_obstacles` (915.2), pero **generaliza mal a `small_loop`** (recompensa negativa, longitud media 296) y **no supera a PPO 20k** en el mapa oculto (915.2 frente a 1118.2). El intento de SAC 50k no se completó (ver nota de la Tabla 1), por lo que no se compara cuantitativamente.

Decisión final. DQN, PPO y SAC comparten arquitectura (`CustomCNN`) y wrappers, lo que hace la comparación justa. Por mejor equilibrio global y mejor resultado en el mapa oculto, el entregable final (`best_agent.zip`) es **PPO 20k**.

8 Conclusión

Se logró un pipeline completo y reproducible (Fase 1 tabular, Fases 2–3 sobre Duckietown) y un agente que generaliza al mapa oculto. La contribución práctica principal fue diagnosticar y resolver el *segmentation fault* nativo mediante *model-first*, sin lo cual el entrenamiento/carga no era viable en Colab. El modelo final (`best_agent.zip`, copia de `ppo_loop_empty_20k_gpu.zip`) supera a las variantes consideradas en el criterio del contrato.

Reproducibilidad

Dependencias: `requirements.txt` (stack exacto) + `gym-duckietown` aparte (`--no-deps`) + re-fijado de `numpy==1.23.5` (detalle en `COLAB_SETUP.md`). Protocolo y tabla de experimentos en `EXPERIMENTS.md`. Comandos principales:

Entrenamiento (CPU o GPU)

```
python scripts/run_training_plan.py --stage ppo20k --execute \  
    --eval-after --init-order model-first
```

Evaluación del modelo final

```
python eval.py --algo ppo --model models/best_agent \  
    --map Duckietown-loop_empty-v0 --init-order model-first --device cpu
```